Week 4: Linux Scheduler Observability

eBPF for Critical Path Analysis

> You can't improve what you can't *measure*, and you can't measure what you can't *observe*.

Today's plan

Seminar (90 min)

- Tracing methodology: what is an event? what is a metric?
- Tracepoints vs kprobes (stability + overhead)
- Observer effect (measurement overhead)

Lab (90 min)

- **SchedLab**: observe scheduler events with eBPF
- Compute **scheduling latency percentiles**
- Basic fairness experiment

Quick environment check (5 minutes)

```
# 1) Does bpftrace run?
sudo bpftrace -e 'BEGIN { printf("eBPF works!\n"); exit(); }'

# 2) Does BTF exist? (helps CO-RE / modern tooling)
ls /sys/kernel/btf/vmlinux

# 3) Kernel version (important for report + behavior)
uname -r
```

If (1) fails: likely permission / missing package.

What (1) does:

- `BEGIN { ... }` runs once at start (no probes attached)
- prints a line and exits immediately
- good for verifying permissions + basic BPF functionality

Part A — Linux scheduler (CFS → EEVDF)

By the end of Part A, you should be able to answer:

- When a task wakes up, **what path does it take before it runs?**
- If p99 scheduling latency is high, **which mechanisms could explain the tail?**
- What changed from **CFS** to **EEVDF** (at a high level)?

What a scheduler optimizes (and what it cannot)

CPU scheduling is a **trade-off** among:

- **Throughput**: keep CPUs busy doing useful work
- **Latency**: run newly-woken work quickly (especially interactive)
- **Fairness**: distribute CPU time by policy (weights, cgroups)
- **Overhead**: avoid too many context switches / migrations

Important limitation:

> The scheduler cannot create CPU time. If all CPUs are busy, someone must wait.

That waiting time is what we measure as **scheduling latency**.

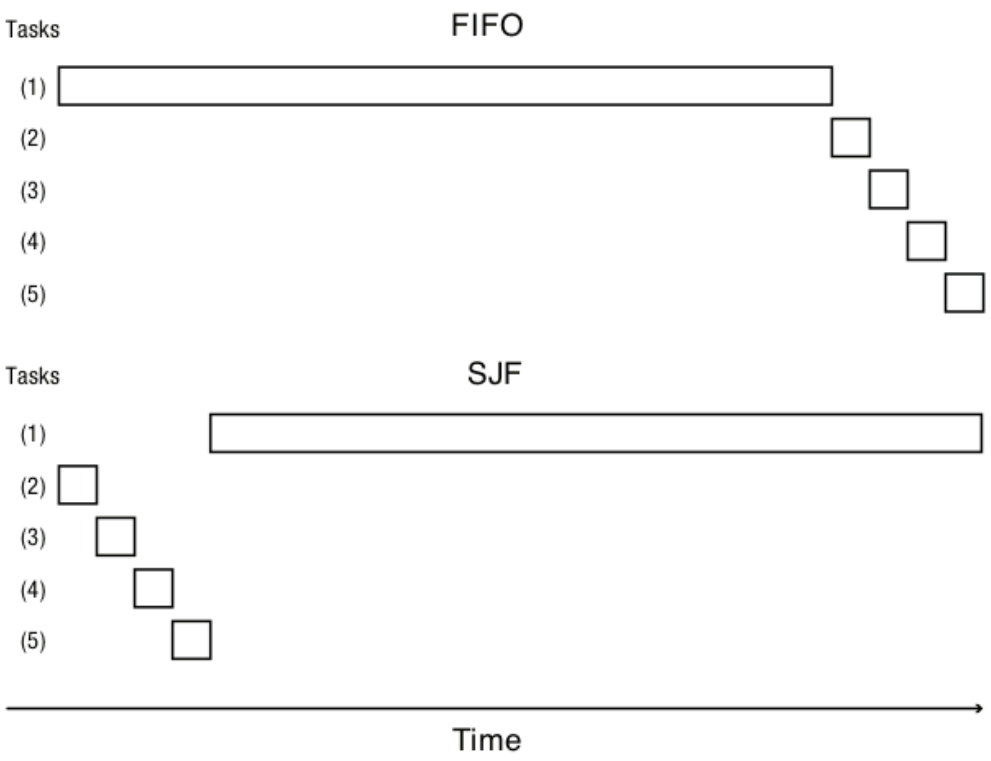
Scheduling policies 101

Classic uniprocessor policies (you should recognize these):

Policy	Core idea	Strength	Common failure mode
FIFO / FCFS	run in arrival order	minimal overhead	convoy effect (short job stuck behind long job)
SJF	run shortest job first	minimizes *average* response time	starvation; needs job-length prediction
RR	fixed time quantum; rotate	avoids starvation	wrong quantum hurts latency or overhead

Modern Linux (CFS/EEVDF) is not “just RR”, but **time quantum + fairness** ideas still show up.

FIFO: convoy effect

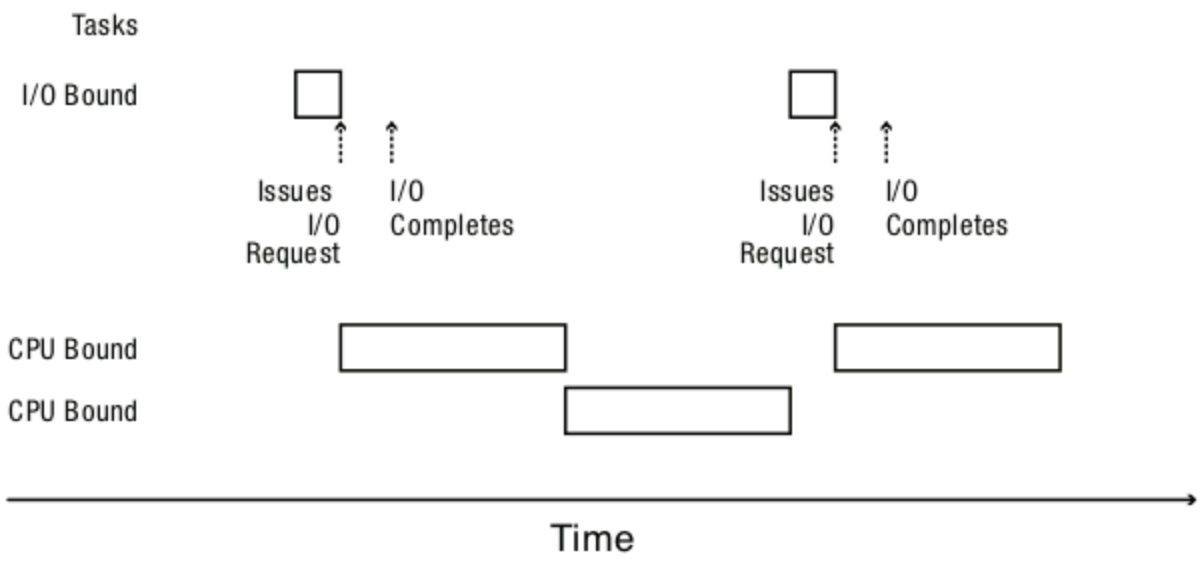


Takeaway:

- FIFO is simple, but one long CPU burst can delay everyone behind it.
- In tail-latency terms: one “elephant” can dominate p99 for “mice”.

RR: time quantum trade-off

If an I/O-bound task needs *1ms CPU* to issue the next I/O, it can still wait behind CPU-bound tasks.



Takeaway:

- RR prevents starvation, but can still create **large wakeup→run delays** when the runqueue is long.
- This is exactly the symptom we will measure with `sched_wakeup` + `sched_switch`.

Case Study: Multi-Level Feedback

Most (previous generation) commercial OSes, including Windows, MacOS, and Linux, use a scheduling algorithm called **multi-level feedback queue (MFQ)**. MFQ is designed to achieve several simultaneous goals:

- Responsiveness. Run short task quickly, as in SJF.
- Low overhead: Minimize the number of preemptions and the time spent making scheduling decisions.
- Starvation-Freedome: All tasks should make progress, as in Round Robin.
- Background Tasks: Defer system maintenance tasks, such as disk defragmentation, so they do not interfere with user work.
- Fairness. Assign processes approcimately their max-min fair share of the processor.

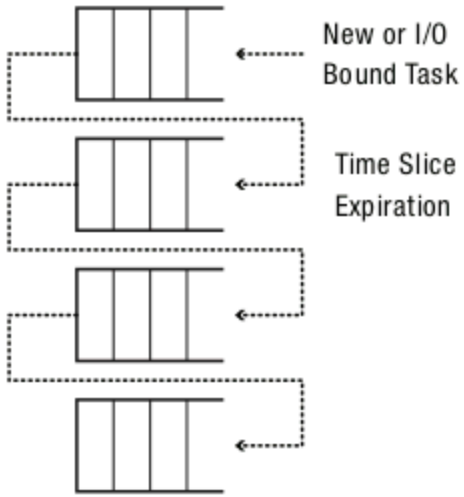
MFQ is an extension of RR

- Instead of only a single queue, MFQ has multiple RR queues, each with a different priority level and time quantum.
- Tasks at a higher priority level preempt lower priority tasks
- Tasks at the same level are scheduled in Round Robin fasion
- Higher priority levels have shorter time quanta than lower levels
- Tasks are moved between priority levels to favor short tasks over long ones.
- A new task enters at the top priority level.
- Task uses up its time quantum, it drops a level
- Task yields the processor, it stays at the same level

MFQ

Priority	Time Slice (ms)
1	10
2	20
3	40
4	80

Round Robin Queues



- A compute-bound task will start as high priority, but it will quickly exhaust its time quantum and fall to the next lower priority, and then the next.
- An I/O-bound task need only a modest amount of computing and hence always be scheduled quickly.

From MFQ to CFS/EEVDF

Many OSes use **multi-level feedback queue (MLFQ/MFQ)** ideas:

- favor interactive / I/O-bound tasks (they sleep often)
- gradually push long CPU-bound tasks to lower priority queues

Modern Linux CFS/EEVDF achieves similar goals with a different mechanism:

- use **weights** (`nice`, cgroups)
- track **service received** (`vruntime`, deadlines)
- keep the system responsive *without* manually moving tasks across many queues

What Linux actually schedules

Linux schedules **tasks** (kernel `struct task_struct`):

- processes and threads are both tasks
- each task belongs to a **scheduling class**

Common classes (conceptual order):

- **STOP/idle** (fallback)
- **CFS** (normal tasks, most of what you run)
- **RT** (`SCHED_FIFO`, `SCHED_RR`)
- **Deadline** (`SCHED_DEADLINE`)

This week's lab focuses on **CFS** behavior and observability.

Kernel-realistic view: scheduling classes (who wins?)

Internally, Linux walks a **scheduling class chain** on each CPU:

Scheduling class chain (conceptual): pick_next_task()

Rule of thumb:

Walk classes from highest → lowest; the first class with runnable tasks on this CPU wins.

Most user processes run in **fair** (CFS/EEVDF). RT/Deadline can preempt CFS.

Two important consequences:

- CFS is the default, but **RT/Deadline can preempt** it
- "Why didn't my task run?" sometimes means "a *different class* took the CPU"

One CPU, one runqueue

Think per CPU core. Key consequence:

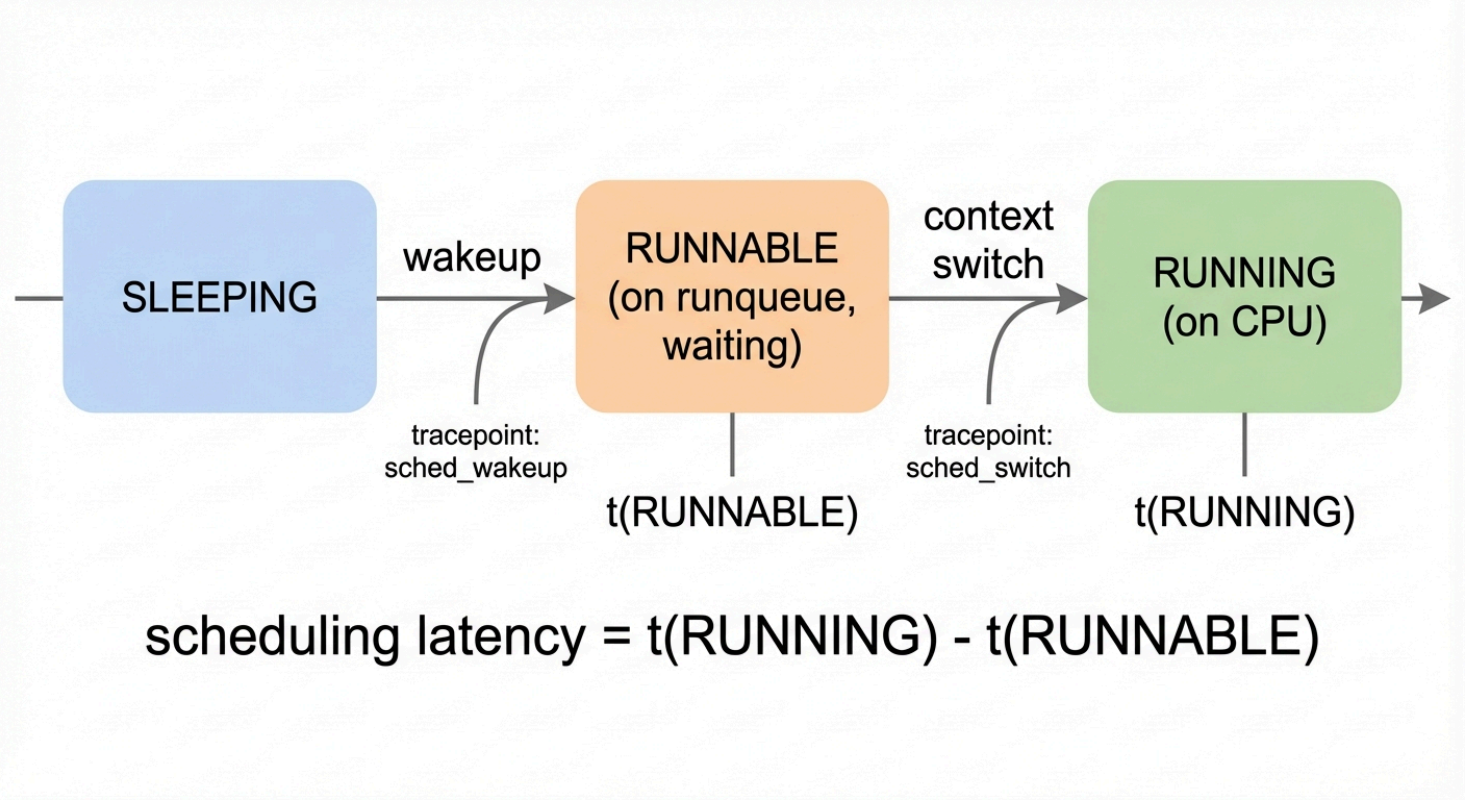
- a task can be “ready” (**RUNNABLE**) but still not “running” (**on-CPU**)

That waiting time is what we often *feel* as latency.

Also remember:

- Linux is **per-CPU** first (local runqueue), then **balances** across CPUs
- migrations can reduce waiting time, but may hurt cache locality

Task state timeline



Scheduling latency (our key metric) =

- `time(task becomes RUNNABLE)` → `time(task becomes RUNNING)`

In tracing terms, we typically estimate this with:

- `sched:sched_wakeup` (becomes runnable)
- `sched:sched_switch` (actually runs)

The critical path: wakeup → enqueue → pick → switch

When a task wakes up, conceptually:

1. **Wakeup**: task transitions to RUNNABLE
2. **Enqueue**: placed into a CPU's runqueue (local or after migration)
3. **Pick**: scheduler selects next task to run on this CPU
4. **Switch**: context switch happens

Tracepoints that correspond to these steps:

- `sched_wakeup` / `sched_wakeup_new`
- `sched_switch`
- (optional) `sched_migrate_task`

What makes scheduling latency large

A waking task often waits because:

1. **CPU is busy**: current task keeps running until a preemption point
2. **Runqueue is long**: many runnable tasks are ahead “by policy”
3. **Wrong CPU**: task wakes on a busy CPU and migrates later (or not)
4. **Overhead / interference**: interrupts, kernel threads, contention

The simplest predictor is still:

- **how many runnable tasks share the CPU** (and their weights)

CFS in one sentence

For normal tasks, Linux uses **CFS (Completely Fair Scheduler)**:

> Give each task CPU time **proportional to its weight**, and keep tasks from falling too far behind.

Where weights come from:

- `nice` (per-task)
- cgroup CPU controller (per-group)

Nice → weight → CPU share (the lever you can control)

Rule of thumb:

- higher weight (lower nice) → larger CPU share → typically lower waiting time

nice	weight (approx)	relative share (vs nice=0)
-20	88761	~86×
0	1024	1×
+19	15	~1/68×

We will use this lever to create controlled “unfairness” in the lab.

CFS mechanism: `vruntime` (weighted time)

CFS tracks a per-task notion of "how much CPU you've gotten", called `vruntime`.

- **Run longer, Higher weight**
- `vruntime` increases *more slowly* for the same wall time

$$vruntime_i = vruntime_i + \frac{weight_0}{weight_i} * runtime_i$$

$$time_slides_k = \frac{weight_k}{\sum_{i=0}^{n-1} weight_i} * sched_latency$$

Interpretation:

> Smaller `vruntime` ≈ "more under-served" (owed CPU time).

`vruntime` intuition example

One CPU, three runnable tasks: A / B / C.

- A: nice=0 (weight≈1024), has been running for a while → large `vruntime`
- B: nice=0 (weight≈1024), just woke up → small `vruntime`
- C: nice=+10 (smaller weight); for the same 1ms, `vruntime` increases faster

Who is more likely to run *next*?

- **B** (smallest `vruntime`, under-served)
- then A
- C has smaller weight → `vruntime` catches up faster → gets a smaller share

This is “weighted fairness”: the same 1ms is not equal under different weights.

Timeslices and preemption

CFS does **not** run a task forever. It computes a target slice based on:

- runnable population (how many tasks are competing)
- tunables like `sched_latency_ns` and `sched_min_granularity_ns`

Wakeup behavior (intuition):

- if a waking task is sufficiently under-served, it may **preempt sooner**
- but if the CPU is overloaded, preemption points still have limits

From CFS to EEVDF

Newer kernels moved CFS closer to an **EEVDF-style** selection rule.

Why change?

- classic CFS “pick smallest `vruntime`” + slice heuristics can create undesirable latency patterns
- EEVDF provides a more direct way to reason about **service deadlines**

Keep this high-level mapping:

- `vruntime` still represents “service received”
- EEVDF introduces the ideas of **eligibility** and **(virtual) deadlines**

EEVDF intuition

EEVDF-style scheduler intuition:

- **eligible**: tasks that are allowed to compete now
- **deadline**: a target time by which a task should get service

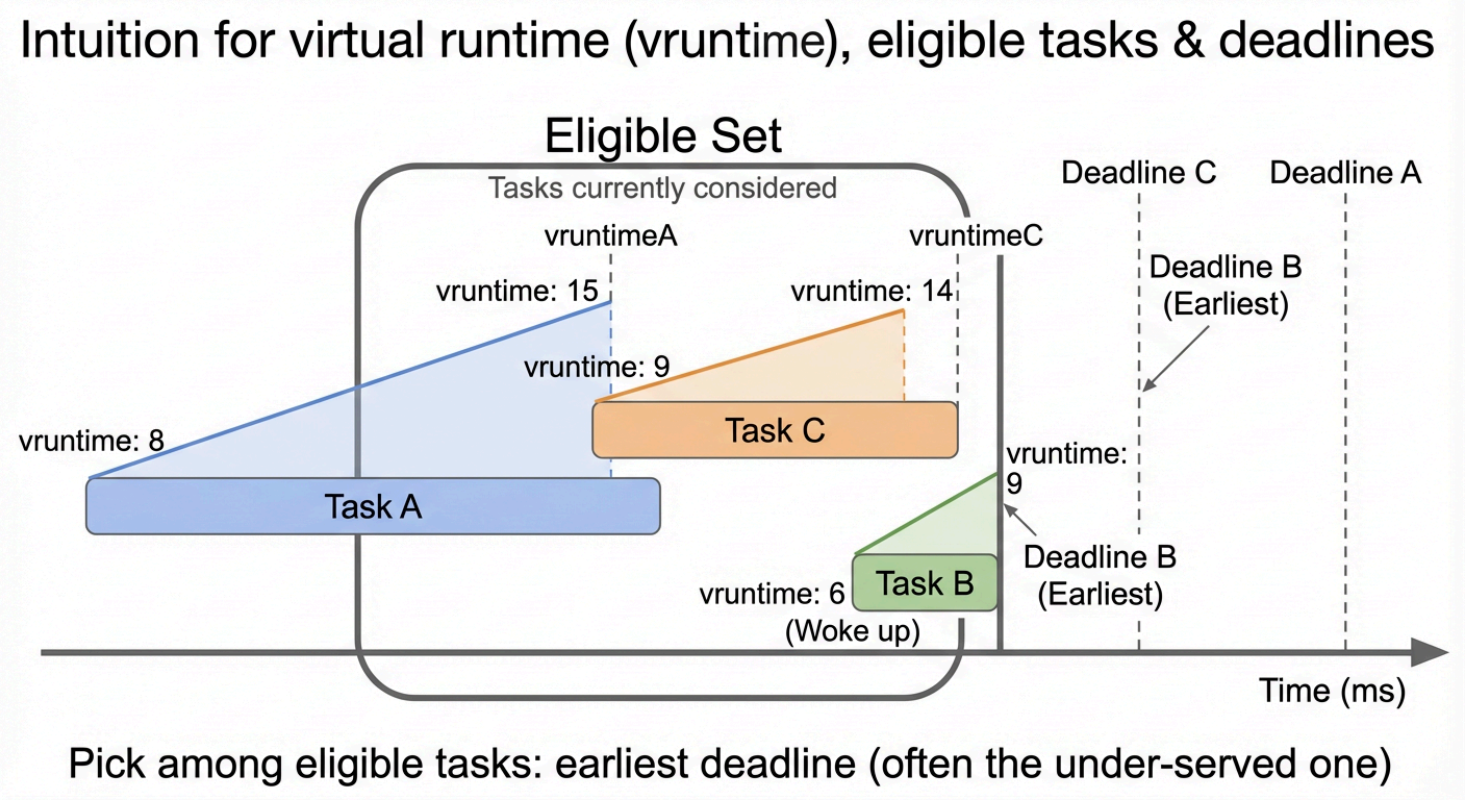
Rule of thumb:

> Among eligible tasks, run the one with the **earliest deadline**.

This helps explain:

- why newly woken interactive work can run quickly
- why an overloaded runqueue still pushes deadlines out (p99 rises)

Visual intuition



A concrete latency example

Assume one CPU, tasks A/B/C are all RUNNABLE.

- if each effectively gets ~2ms before switching,
- and your task wakes up behind 2 tasks,

then *even with a perfect scheduler* you can see ~4ms wakeup→run latency.

Now add tail amplifiers:

- interrupts, kernel threads, migrations, contention

That's typically where p99 comes from.

What you should remember for the lab

You only need a few anchors:

- wakeup→run latency is primarily **queueing delay** on a CPU
- CFS uses **weights** and a **service counter** (`vruntime`)
- modern kernels use an **EEVDF-style** earliest-deadline selection among eligible tasks

Everything else in this week is about:

> turning those concepts into **events → metrics** using tracepoints.

What exactly do we observe?

What you want	What it means	Tracepoints to use
Scheduling latency	runnable → running delay	`sched_wakeup` + `sched_switch`
Context switch rate	overhead / churn	`sched_switch`
Fairness (rough)	who gets CPU vs waits	`sched_switch` (+ optional runtime counters)
Migration	cache/cpu locality disruption	`sched_migrate_task` (optional)

This mapping is the “design pattern” for tracing.

Part B — eBPF, but we'll write bpftrace one-liners

Goal: in this course, eBPF is mainly a **microscope**.

- in-class demos: **bpftrace one-liners** (run immediately, see results immediately)
- lab/project: SchedLab is more structured (CSV export, percentile computation)

bpftrace: treat it as “awk for the kernel”

You write a short script:

- pick an event (tracepoint / kprobe)
- write a tiny handler
- use built-in aggregations (`count() / hist() / sum() / avg()`) to get results

Core experience:

> No C, no compilation — first, make the kernel behavior visible.

First: learn to *find events* (tracepoints)

```
# List scheduler-related tracepoints
sudo bpftrace -l 'tracepoint:sched:*'

# Inspect tracepoint fields (arguments)
sudo bpftrace -lv tracepoint:sched:sched_switch
sudo bpftrace -lv tracepoint:sched:sched_wakeup
```

What these commands do:

- `-l 'tracepoint:sched:*'` lists available scheduler **tracepoints** (stable instrumentation points)
- `-lv tracepoint:...` prints the **struct fields** you can access via `args->...`

Why we care:

> Good tracing starts by discovering the exact field names (PID, comm, CPU, etc.) that the kernel exports.

Rule of thumb:

- prefer **tracepoints** (stable, structured arguments)
- only use **kprobes** when tracepoints are not enough

The most common bpftrace “patterns”

In one-liners, you will use these constantly:

- filters: `/ pid == 123 /`, `/ comm == "bash" /`
- state: `@ts[pid] = nsecs`
- aggregation: `@x = count()`, `@h = hist($d)`
- cleanup: `delete(@ts[pid])`

These four are enough for ~80% of course-level observability tasks.

bpftrace semantics cheat-sheet

Probe is an event source:

- `tracepoint:subsys:event { ... }` runs the handler on every event

`args->...` **fields** come from `-lv` output:

- example: `sched_switch` exposes `prev_pid`, `prev_comm`, `next_pid`, `next_comm`, ...

`@name[...]` **is a BPF map** (kernel-side storage):

- `@ts[pid] = nsecs` stores per-PID state
- `delete(@ts[pid])` removes a key
- `clear(@ts)` clears the whole map

Useful built-ins:

- `nsecs` (monotonic time)
- `pid` / `tid` (current process/thread)
- `cpu` (current CPU)

One-liner 1: Who is switching out the most?

```
sudo bpftrace -e '
tracepoint:sched:sched_switch { @sw[args->prev_comm] = count(); }
interval:s:5 { print(@sw, 20); clear(@sw); }
'
```

One-liner 1 explained

Probe:

- `tracepoint:sched:sched_switch` fires on every context switch

Handler logic:

- `@sw[key] = count()` counts how many times each key appears
- `key = prev_comm` means: **who got switched out** (CPU burst ended)

Reporter:

- `interval:s:5` prints every 5 seconds
- `print(@sw, 20)` shows the top 20 keys by count
- `clear(@sw)` resets the window (so you get “per 5 seconds” behavior)

How to interpret:

- high switch-out count often means **short CPU bursts** (I/O, locks, sleep/wakeup churn)
- it does *not* mean “uses the most CPU time” (that would require runtime accounting)

Useful variations:

```
# Count who gets switched *in* instead
tracepoint:sched:sched_switch { @in[args->next_comm] = count(); }
```

One-liner 2: wakeup → running latency histogram

```
sudo bpftrace -e '
tracepoint:sched:sched_wakeup { @ts[args->pid] = nsecs; }
tracepoint:sched:sched_switch / @ts[args->next_pid] / {
  $d_us = (nsecs - @ts[args->next_pid]) / 1000;
  @lat_us = hist($d_us);
  delete(@ts[args->next_pid]);
}
'
```

This is the smallest version of our lab recipe: “two events + one piece of state”.

One-liner 2 explained (wakeup → first run)

What it measures:

- **wakeup→run latency** = time from `sched_wakeup(pid)` until that PID is scheduled as `next_pid` in `sched_switch`

How the state works:

- `@ts[pid] = nsecs` stores wakeup timestamp in a BPF map
- on switch-in, we compute `now - @ts[pid]` and delete the entry

Why divide by 1000?

- `nsecs` is in nanoseconds
- histograms are easier to read in **microseconds** (`$d_us`)

How to read `hist()` output:

- buckets are power-of-two ranges (log2-ish)
- use it to see p50-ish vs tail shape quickly

Common pitfalls (important):

- If a task wakes multiple times before running, `@ts[pid]` gets overwritten
- If you care about *newly created tasks*, also track `sched_wakeup_new`
- PID reuse is possible on long traces; keep runs short or add extra keys (PID+start time)

One-liner 3: latency by process

```
sudo bpftrace -e '
tracepoint:sched:sched_wakeup { @ts[args->pid] = nsecs; }
tracepoint:sched:sched_switch / @ts[args->next_pid] / {
  $d_us = (nsecs - @ts[args->next_pid]) / 1000;
  @lat_by_comm[args->next_comm] = hist($d_us);
  delete(@ts[args->next_pid]);
}
'
```

If you only care about one command, add a filter:

```
... / args->next_comm == "python" / { ... }
```

One-liner 3 explained (break down the tail)

What changes vs One-liner 2:

- instead of one global histogram, we keep **one histogram per** `comm`:
  - `@lat_by_comm[comm] = hist(latency)`

Why this is useful:

- identifies *which* runnable work tends to suffer high wakeup→run latency
- helps you decide the next step: filter, zoom in, or correlate with runqueue length

Cost / caution:

- if your system has many distinct `comm` values, this map can grow
- in practice, add a filter early (PID, `comm`, or cgroup) when doing longer runs

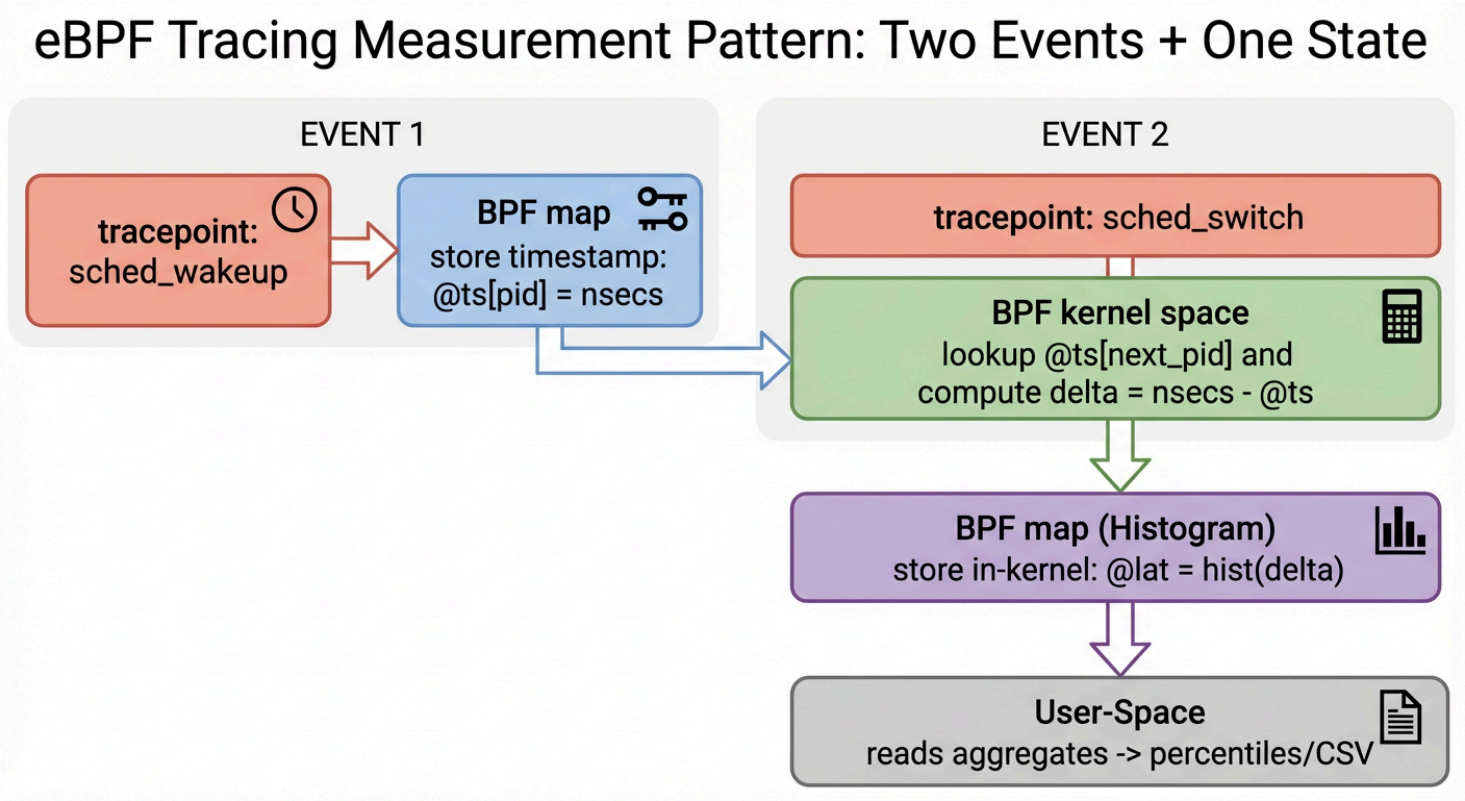
The key measurement pattern (this is the whole lab)

Scheduling latency measurement is “two events + one piece of state”:

1. on `sched_wakeup(tid)` → store `wakeup_time[tid] = now`
2. on `sched_switch(next_tid)` → look up `wakeup_time[next_tid]`
3. latency = `now - wakeup_time[next_tid]`
4. push to histogram / record

This is how you turn kernel events into a distribution.

The recipe, visualized



One-liner 4: who is being woken up constantly?

```
sudo bpftrace -e '
tracepoint:sched:sched_wakeup { @w[args->comm] = count(); }
interval:s:5 { print(@w, 20); clear(@w); }
'
```

Use case:

- if a service thread is woken up constantly but still does not get CPU quickly, your runqueue may be congested

One-liner 4 explained

Probe:

- `sched_wakeup` fires when a task becomes RUNNABLE (is woken)

What the script computes:

- counts wakeups per `comm` in 5-second windows

How to interpret:

- high wakeup count + high wakeup→run latency often indicates:
  - many threads competing (runqueue congestion), or
  - a tight wakeup/block loop (lock contention / polling / thundering herd)

Useful refinement:

```
# Focus on one service
tracepoint:sched:sched_wakeup / args->comm == "myservice" / { @w = count(); }
```

Observer effect

Every tracing tool costs something.

Where overhead comes from:

- running on a hot path (context switches can be **very frequent**)
- map lookup/update
- emitting per-event records

Practical rule for this course:

- **aggregate in-kernel** as much as possible
- keep per-event logic minimal

VM reality: don’t rely on PMU, but tracepoints are solid

What you want	In a VM	Recommendation
cache-miss / branch-miss (hardware events)	often unsupported / always zero	use tracepoints/eBPF for scheduling/blocking, or use `valgrind` as a fallback
scheduler events (wakeup/switch)	usually works	**bpftrace / SchedLab**
flamegraph-style profiling	may be limited	prioritize “waiting vs running” attribution

Part C — SchedLab: how to make the experiments “work”

What we want students to get from the lab:

- a latency distribution under different loads
- a story: “why does p99 move?”

SchedLab tasks (what to focus on)

1. **Event model**
   - which tracepoint fires? what does each argument mean?
2. **Latency distribution**
   - compute p50/p90/p99; compare idle vs loaded
3. **Fairness (basic)**
   - which tasks get CPU? which tasks wait?

The point is not fancy math; it’s connecting mechanism → measurement.

Build + sanity check (do this once)

```
# From repo root
cd schedlab
make

# Quick stream mode: you should see scheduler events
sudo ./schedlab --mode stream
```

If build fails, capture:

- compiler error log
- `uname -r`
- distro version

Task 2: collect latency data (idle vs loaded)

```
# idle-ish
timeout 15s sudo ./schedlab --mode latency --csv --csv-header > idle.csv

# loaded
timeout 20s sudo ./schedlab --mode latency --csv --csv-header > loaded.csv &
stress-ng --cpu 2 --io 2 --timeout 15s &
wait
```

Analysis checklist:

- convert ns → us/ms
- histogram + percentiles
- explain which mechanism likely caused tail

Percentiles: what they mean (fast)

Metric	Meaning
p50	“typical” wakeup latency
p99	“bad but not rare”
p99.9	“extreme tail”

Systems insight:

- **average** hides pain; **tail** reveals contention/rare slow paths.

Task 3: fairness experiment (one simple way)

```
timeout 25s sudo ./schedlab --mode fairness --csv --csv-header > fairness.csv &
stress-ng --cpu 1 --fork 5 --timeout 20s &
wait
```

A crude but useful ratio:

- `CPU_share ≈ run_time / (run_time + wait_time)`

Then ask:

- do weights/nice explain the share?

Common pitfalls (that waste hours)

1. Running inside Docker / restricted environment (BPF blocked)
2. Forgetting `sudo`
3. Recording too long (giant CSV, hard to analyze)
4. Confusing “scheduler latency” with “application latency”

Good habit:

- always write down **your exact latency definition** in the report.

What a good lab report looks like

Must include:

- environment: VM/bare metal, kernel version
- the workload: what load generator, parameters
- results: p50/p90/p99 + histogram
- explanation:
  - *what scheduler mechanism likely explains the tail?*
  - *what did you change between experiments?*

We care more about a correct story than perfect formatting.

Next week preview

Week 5: Container Boundaries — Namespaces and Cgroups

You'll learn how:

- a container gets its own process view
- a cgroup limits CPU/memory

This week's scheduler observability helps you debug:

- "my pod is slow" → is it throttled or just waiting?

Questions?

If something is still fuzzy, ask using one of these forms:

- “What does *RUNNABLE* mean exactly?”
- “Which tracepoint should I use to measure X?”
- “In the latency recipe, why do we need both wakeup and switch?”

Break (5 min)

Then we start Lab 3: SchedLab